

Introdução à Computação

comparando arrays e aritmética de ponteiros

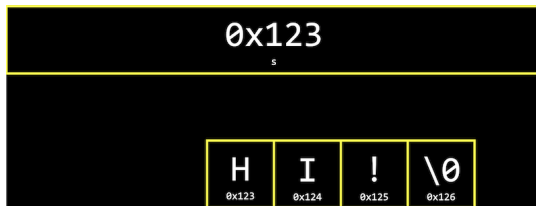
Alexandre Rademaker

Strings I

Como já vimos, strings são arrays de caracteres. Podemos declarar com

```
char s[] = "HI!".
```

Cada caractere, se está armazenado na memória, também tem um endereço único, e `s` é na verdade apenas um **ponteiro** com o endereço do primeiro caractere. Podemos acessar cada caracter como `s[0]` , `s[1]` etc.



Strings II

`s` é uma variável do tipo **`char*`**, que é um ponteiro para um caractere.

Lembre-se de que podemos ler a string inteira começando no endereço em `s` e continuar lendo um caractere por vez da memória até chegarmos a `\0`.

Strings III

address02.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 typedef char* string;
5
6 int main(int argc, char *argv[]) {
7
8     char s[] = "Hello1";
9     char *t = "Hello2", *u = "Hello3";
10    string v = "Hello4", x = "Hello5"; // typedef is
        convenient
11
12    printf("'s' at %p has size %lu\n", t, t, strlen(t));
13    printf("'s' at %p has size %lu\n", v, v, strlen(v));
14 }
```

Strings IV

Quando escrevemos `typedef char *string;` estamos criando um tipo de dados personalizado para a palavra `string`, tornando-a equivalente a `char *`.

Aritmética dos ponteiros I

Agora que sabemos o que são strings. Podemos imprimir um caractere por vez com

address03.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     char *s = "HI!";
6     printf("%c at %p\n", s[0], &s[0]);
7     printf("%c at %p\n", s[1], &s[1]);
8     printf("%c at %p\n", s[2], &s[2]);
9     printf("\\0 at %p\n", &s[3]);
10 }
```

Quando declaramos uma string com aspas duplas, o compilador decide como colocar esses caracteres na memória como um array.

Aritmética dos ponteiros II

Mas podemos usar os endereços diretamente

address04.c

```
1 #include <stdio.h>
2
3 int main(void) {
4     char *s = "HI!";
5     printf("%c = %c = %c\n", *s, *(s + 0), s[0]);
6     printf("%c = %c\n",      ,      *(s + 1), s[1]);
7     printf("%c = %c\n",      ,      *(s + 2), s[2]);
8 }
```

`*s` lê o endereço armazenado em `s`, e `*(s + 1)` o endereço na memória 1 byte (char) maior.

`s[1]` é um **syntactic sugar**, uma abstração mais legível para humanos para a expressão `*(s + 1)`.

Aritmética dos ponteiros III

Pointer arithmetic é o processo de aplicar operadores matemáticos à ponteiros, tratando-os como números.

comparando arrays I

podemos comparar números

compare0-intro.c

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     int i = 12;
6     int j = 45;
7
8     printf("%s\n", i == j ? "same" : "different");
9 }
```

mas para comprar strings?

comparando arrays II

compare1-intro.c

```
1 #include <stdio.h>
2
3
4 int main(void) {
5
6     char s[100];
7     char t[100];
8
9     printf("s: "); scanf("%s", s);
10    printf("t: "); scanf("%s", t);
11
12    // so compiler don't see s == t
13    char *a = s;
14    char *b = t;
15
16    // always false!!!
17    printf("%s\n", a == b ? "same" : "different");
18    // only first char
19    printf("%s\n", *a == *b ? "same" : "different");
20 }
```

Já vimos que para comparar strings precisamos da `strcmp` !

comparando arrays III

compare2-intro.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void) {
5
6      char s[100];
7      char t[100];
8
9      printf("s: "); scanf("%s", s);
10     printf("t: "); scanf("%s", t);
11
12     // print strings
13     printf("%s at %p\n", s, s);
14     printf("%s at %p\n", t, t);
15     printf("%s\n", strcmp(s,t) == 0 ? "same" : "different");
16 }
```

comparando arrays IV

Cada string é um ponteiro para um endereço diferente? Não exatamente.

compare3.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(void) {
5
6     char *s = "hello";
7     char *t = "hello";
8
9     printf("s = %p  t = %p\n", s, t);
10    printf("%s\n", strcmp(s,t) == 0 ? "same" : "different");
11 }
```

comparando arrays V

Será que podemos implementar nossa própria `strcmp`? Como seria?

1. recebe dois ponteiros para char e devolve um `int`
2. ler [documentação](#) e entender que a função deve retornar 0, algum positivo ou algum negativo.
3. um loop 'andando' pelos arrays recebidos até que algum `0` seja encontrado.

Veja `compare4-intro.c`

comparando arrays VI

Em C, também podemos obter o endereço de `s` e `t` e armazenar em uma variável do tipo `char **`, um ponteiro para um ponteiro.

copiando arrays I

copy0-scanf.c

```
1 #include <ctype.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int main(void) {
6     char *s;
7     char *t = s;
8
9     printf("s:"); scanf("%s", s);
10
11     // capitalize first letter in string
12     if (strlen(s) > 0) {
13         s[0] = toupper(s[0]);
14     }
15
16     printf("s: %s\n", s);
17     printf("t: %s\n", t);
18 }
```

Ambas as saídas foram capitalizadas, por que?